Day 2 — Agents and Tool Calling

Wednesday 14 Oct 2026 · AWS AIP-C01 exam task covered: **2.1 Agentic AI and tool integrations (part A)** · Exam: Sat 7 Nov

Today in one paragraph
An agent is a model plus instructions, tools and memory running in a **loop**: it decides at runtime which tool to call, observes the result and repeats until it can answer. The model never executes a tool itself — it only requests one, and your code (or Bedrock Agents' Lambda) runs it. Bedrock Agents packages this loop (action groups, return of control, memory, aliases), but it is now **Agents Classic in maintenance mode** and blocked for new accounts, so the hands-on path is the Converse API tool loop today and AgentCore / Strands tomorrow.

1. Warm-up results (Day 1 recall) — 3.5 / 5

#	Question	Your answer	Verdict and correct answer
1	KB quick-create vector store; why delete after lab?	OpenSearch; billed per hour even when idle	✓ Correct. Precisely: **OpenSearch Serverless**, billed in OCU-hours with a minimum.
2	Can chunking be changed after creation?	No — create a fresh data store	½ Idea right, term wrong: create a new **data source** (or new KB) and **re-sync**.
3	'error E-4031' returns irrelevant chunks	Hybrid search	✓ Correct — the keyword (BM25) side matches the exact token.
4	Guardrails feature for answers not supported by retrieved docs	Metadata filtering during prompt augmentation	✗ Wrong. Answer: **Guardrails contextual grounding check**. Also, metadata filtering happens at **retrieval**, not augmentation.
5	Two parsing options for table-heavy PDFs	Bedrock Data Automation; FM parsing	✓ Correct (Textract also valid for scanned pages).

2. What an agent is

```
AGENT = model + instructions + tools + memory + a LOOP
```

Approach	What it does	Who decides the steps
Plain LLM call	Prompt in, answer out	Nobody — single step
RAG	Retrieve once, then answer	You (fixed)
Workflow (Step Functions, Bedrock Flows)	Predefined sequence of steps	You, in advance
Agent	Chooses at runtime which tool, how often, in what order	**The model**

ReAct loop (Reason + Act) — the default Bedrock Agents orchestration:

```
User: "Cancel my order 1001 if it hasn't shipped"
  Thought: need the status first        -> Action: get_order_status(1001)
  Observation: SHIPPED
  Thought: shipped, cannot cancel       -> Final answer
```

Exam rule
Known, fixed steps → **workflow** (cheaper, deterministic, auditable). Path depends on intermediate results → **agent**.

3. Tool calling — the mechanics

1. You send the model tool definitions: **name, description, JSON schema** of parameters.
2. The model replies with a tool request — Converse API: **stopReason = "tool_use"** plus a **toolUse** block (name, input, toolUseId).
3. **Your code executes the tool.** The model never runs anything.
4. You send a **toolResult** (same toolUseId) back as a user message; the model continues the loop.

- Tool **descriptions are prompts**: vague description → wrong tool chosen. First fix is better descriptions + instructions, not a bigger model.
- Build the loop yourself with **Converse API + toolConfig**, or let **Bedrock Agents** (Classic) / **AgentCore** run it.

4. Bedrock Agents (Classic) — components

Component	What it is	Exam angle
Model + instructions	The FM the agent reasons with, plus its role/rules in natural language.	Instructions shape tool use.
Action groups	The tools. Defined by an **OpenAPI schema** or **function details**.	One action group can hold several functions.
↳ Lambda executor	Bedrock invokes your Lambda with the parameters.	Most common.
↳ Return of control	Bedrock returns the parameters **to your application**; your app executes and sends results back on the next InvokeAgent call.	Action must run in the app's context (authenticated session, on-prem, long-running, UI).
Built-in action groups	**User input** (ask clarifying questions), **Code Interpreter** (write/run code: maths, charts, files).	'Agent must ask for missing info' → User input.
Knowledge Bases	Attach a KB with a description of when to use it.	Agent decides when to retrieve.
User confirmation	**requireConfirmation** on a function — asks before executing.	Destructive actions.
Guardrails	Attach a guardrail to filter inputs/outputs.	Day 6.
Memory	**sessionAttributes** (whole session), **promptSessionAttributes** (single turn); **long-term memory** = session summaries kept for N days under a memoryId.	'Remember users across sessions' → long-term memory.
Advanced prompts	Override pre-processing, orchestration, KB-response and post-processing templates; custom orchestration Lambda.	Fine control.
Trace	**enableTrace** shows rationale, invocation inputs, observations.	How to debug 'why did it do that?'
Versions and aliases	Edit **DRAFT** → Prepare → **version** → **alias** points at a version. Apps call InvokeAgent(agentId, aliasId).	Rollback = repoint alias. Production never calls DRAFT.

Two permissions that trip people up

1. The **agent service role** needs bedrock:InvokeModel (and KB Retrieve if a KB is attached).
2. The **Lambda needs a resource-based policy** allowing **bedrock.amazonaws.com** to invoke it. 'Access denied calling the Lambda' → this is missing.

5. Lambda contract (function details)

Bedrock sends:

```
{ "actionGroup": "OrderActions", "function": "get_order_status",
```

```
  "parameters": [{"name": "order_id", "type": "string", "value": "1002"}],
  "sessionAttributes": {}, "promptSessionAttributes": {} }
```

Your Lambda must return exactly this shape:

```
{ "messageVersion": "1.0",
  "response": { "actionGroup": "OrderActions", "function": "get_order_status",
    "functionResponse": { "responseBody": { "TEXT": { "body": "Order 1002: PROCESSING" } } } } }
```

OpenAPI-schema variant: input has **apiPath** and **httpMethod**; response has **httpStatusCode** and **responseBody["application/json"]**.

6. Decision rules

1. Fixed, known steps → **Step Functions / Bedrock Flows**, not an agent.
2. Managed agent, least effort → Bedrock Agents (Classic) / **AgentCore**. Full control → **Converse API with toolConfig**.
3. Action must run inside the client app → **return of control**.
4. Agent must ask for missing details → **User input**.
5. Destructive action → **user confirmation** + guardrail.
6. Wrong tool chosen → improve **descriptions and instructions**; verify with the **trace**.
7. Remember users across sessions → **long-term memory**.
8. Safe release and rollback → **aliases** pointing at versions.
9. Agent can't invoke Lambda → Lambda **resource-based policy**.
10. Maths / data analysis / charts → **Code Interpreter**.

Azure mapping: Foundry Agent Service = agent + **threads / messages / runs**; tools include function calling, OpenAPI, Azure Functions, AI Search, file search, code interpreter. A run in status **requires_action** is Azure's return of control.

7. Industry update — Bedrock Agents Classic maintenance mode

- Bedrock Agents is now **Bedrock Agents Classic**, in **maintenance mode** since **30 July 2026**.
- Accounts without Agents usage in the previous 12 months get **403** on **CreateAgent** and **InvokeInlineAgent**. No exception process — this is why your lab was blocked.
- Knowledge Bases, Guardrails and models are **not affected**.
- AWS's recommended path: **Amazon Bedrock AgentCore** (managed harness, Runtime, Gateway, Memory, Identity, Observability).
- **Exam impact:** the AIP-C01 guide predates this; Task 2.1 names **Strands Agents, AWS Agent Squad, MCP and Step Functions**. Know Agents Classic concepts and AgentCore.

8. Lab work

Step	Status
Planned: OrderHelper in Bedrock Agents console (action group + Lambda + confirmation + alias)	Blocked — 'Bedrock Agents is in Maintenance Mode' (new account).
Replacement: OrderHelper with the Converse API tool loop in Python	**Pending** — waiting for the account's on-demand tokens-per-minute (TPM) quota for the model.

Replacement lab code (run it once the quota is available):

```
import boto3

brt = boto3.client("bedrock-runtime", region_name="us-east-1")
```

```
MODEL_ID = "us.amazon.nova-lite-v1:0"   # check the exact ID / inference profile in the console

ORDERS = {"1001": "SHIPPED", "1002": "PROCESSING", "1003": "DELIVERED"}

def get_order_status(order_id):
    return f"Order {order_id} status: {ORDERS.get(order_id, 'NOT FOUND')}"

def cancel_order(order_id):
    # Human-in-the-loop: what "requireConfirmation" does in Bedrock Agents
    if input(f"!! Confirm cancel order {order_id}? (y/n): ").lower() != "y":
        return "User declined the cancellation."
    if ORDERS.get(order_id) == "PROCESSING":
        ORDERS[order_id] = "CANCELLED"
        return f"Order {order_id} cancelled."
    return f"Order {order_id} cannot be cancelled (status: {ORDERS.get(order_id, 'NOT FOUND')})."

TOOLS = {"get_order_status": get_order_status, "cancel_order": cancel_order}

def spec(name, desc):
    return {"toolSpec": {"name": name, "description": desc,
            "inputSchema": {"json": {"type": "object",
                "properties": {"order_id": {"type": "string",
                                            "description": "The order number, e.g. 1002"}},
                "required": ["order_id"]}}}}

TOOL_CONFIG = {"tools": [
    spec("get_order_status", "Returns the current status of a customer order."),
    spec("cancel_order", "Cancels an order. Only PROCESSING orders can be cancelled."),
]}
SYSTEM = [{"text": "You are an order support assistant. Use tools to check or cancel orders. "
                   "Never guess a status. If the order number is missing, ask for it."}]

def ask(question):
    messages = [{"role": "user", "content": [{"text": question}]}]
    while True:                                          # <-- the agent LOOP
        resp = brt.converse(modelId=MODEL_ID, system=SYSTEM,
                            messages=messages, toolConfig=TOOL_CONFIG)
        msg = resp["output"]["message"]
        messages.append(msg)
        print(f"[stopReason={resp['stopReason']}]")
        if resp["stopReason"] != "tool_use":
            return "".join(b.get("text", "") for b in msg["content"])
        results = []
        for block in msg["content"]:
            if "toolUse" in block:
                tu = block["toolUse"]
                print(f"  -> model requested {tu['name']}({tu['input']})")   # your "trace"
                out = TOOLS[tu["name"]](**tu["input"])                       # YOUR code executes
                print(f"  <- observation: {out}")
                results.append({"toolResult": {"toolUseId": tu["toolUseId"],
                                               "content": [{"text": out}]}})
        messages.append({"role": "user", "content": results})

if __name__ == "__main__":
    while (q := input("\nYou: ")) not in ("exit", "quit"):
        print("Agent:", ask(q))
```

Test prompts: status of 1002 · cancel 1002 (confirmation fires) · cancel 1001 (refused) · 'cancel my order' (asks, no tool call) · two questions in one message (two tool calls). Stretch: move get_order_status into a Lambda and call it with boto3; make descriptions vague and watch the wrong tool get picked.

9. Mistakes and learnings

What happened	Correction / learning
Answered the grounding question with metadata filtering.	**Grounding** = is the answer true to the sources → Guardrails contextual grounding check. **Filtering** = who may see which sources → metadata filters at retrieval. The exam pairs them to confuse you.
Said 'data store' for a KB data source.	Use exact service vocabulary: **data source**, **sync / ingestion job**.
Wrote 'Bedrock work flows'.	The service is **Amazon Bedrock Flows**.
Lab blocked: Agents Classic maintenance mode.	New accounts can't call CreateAgent / InvokeInlineAgent. Use Converse tool loop or AgentCore.
Lab delayed: model TPM quota not yet available.	New accounts can start with low or zero on-demand quotas. Check **Service Quotas → Amazon Bedrock** (tokens/requests per minute per model). Exceeding quota → **ThrottlingException**; fix with retries + exponential backoff, a quota increase, cross-region inference profiles or another model (exam Tasks 2.4 and 5.2).

10. Day 2 quiz — your answers graded (5 / 5)

#	Question	Your answer	Verdict
1	Bank transfer must use the app's authenticated session; no backend creds in AWS.	Return of control	✓ The action executes inside the app, so credentials never leave it.
2	Agent calls get_invoice for refund questions; get_refund_status exists. First fix?	Sharpen function descriptions; instruct the agent to always use tools	✓ Descriptions first, instructions second; confirm with the **trace**.
3	Production broke after a change; need instant rollback.	Versions + aliases; test on DRAFT	✓ Rollback = **repoint the alias** to the previous version — no client change. Production never calls DRAFT.
4	Agent role has lambda:InvokeFunction but still access denied.	Lambda resource-based policy for bedrock.amazonaws.com	✓ Correct.
5	Fixed claims process: extract → validate → store → notify.	Not an agent; Step Functions or Bedrock Flows	✓ Correct — deterministic, auditable, cheaper. (Name: **Bedrock Flows**.)

11. Key takeaways

- Agent = model + instructions + tools + memory + loop; the model only requests tools.
- Converse API: stopReason **tool_use** → run tool → send **toolResult** with the same toolUseId.
- Bedrock Agents Classic vocabulary is still examinable: action groups, return of control, User input, Code Interpreter, confirmation, memory, trace, versions/aliases, Lambda resource policy.
- Workflow when steps are known; agent when the path depends on results.
- Quotas and throttling are real — and exam material.

12. Warm-up for Day 3 — answer from memory (answers in the Day 3 note)

1. In the Converse API, which stopReason tells you the model wants a tool, and who executes the tool?
2. Name the two ways to define the tools of a Bedrock Agents action group.
3. Your agent must ask the user for a missing order number before acting. Which built-in capability enables this?
4. What is the difference between sessionAttributes and promptSessionAttributes?
5. Which two Bedrock Agents Classic operations are blocked for new accounts, and what does AWS recommend instead?

Next: Day 3 — Strands Agents, Agent Squad, MCP and AgentCore (Thu 15 Oct)
Task 2.1 part B: building agents with Strands, multi-agent with Agent Squad, tools over MCP, AgentCore Runtime / Gateway / Memory / Identity / Observability, Step Functions for agent workflows.